ADVANCED DATABASE MANAGEMENT SYSTEMS (ADBMS)

SHOPEASE

Database Design and Implementation Using MongoDB

A Full Stack E-Commerce Web Application

ADBMS Mini Project Report

Technology: MongoDB with Mongoose | Node.js + Express.js | React.js

Subject	Advanced Database Management Systems (ADBMS)
Semester	5B div
College / University	L.D College of Engineering
Academic Year	2025-2026

Sr. No.	Group Member Name	Enrollment No.
1	Vasava Vinit	240280116149
2	Vasava Meet	240280116148
3	Vasava Vatsal	240280116151

1. Abstract

ShopEase is a full-stack e-commerce web application whose data is stored in MongoDB and accessed through Mongoose. The application lets customers register, log in, browse products, use a cart and wishlist, check out and track orders, and lets administrators manage products and orders. This report studies ShopEase from the database point of view. It explains why a document database was chosen, how the data is modelled into the Users, Products and Orders collections, how relationships are handled, how every application feature maps to a database operation, and how querying, aggregation, indexing, transactions and security apply to the system. The same application was submitted earlier as a Web Application Development (WAD) project; this report concentrates on its database layer.

2. Introduction

An e-commerce system works with several kinds of data at the same time: customer accounts, a product catalogue, wishlists and orders. This data must be stored permanently, retrieved quickly, kept consistent and protected from unauthorized access. These are the central concerns of database management.

ShopEase stores its data in MongoDB, a document-oriented NoSQL database, and uses Mongoose schemas to define the structure of each collection. The React frontend and the Express backend exchange JSON, which matches the JSON-like (BSON) documents that MongoDB stores.

3. Problem Statement

Traditional small-scale product selling systems may not provide an integrated platform for product browsing, customer accounts, cart management and order administration. When this information is kept in separate files or sheets, data gets duplicated, searching is slow, nobody controls who can change it, and there is no dependable record of orders. ShopEase addresses this by keeping users, products and orders in one database, with defined structure, controlled access and queryable history.

4. Objectives

- Design a suitable data model for users, products and orders in MongoDB.
- Represent the relationships between entities using references and embedded fields.
- Show how every application feature maps to a database (CRUD) operation.
- Demonstrate queries and aggregation pipelines for searching and reporting.
- Explain indexing, transactions, concurrency and security for the database.
- Demonstrate REST API communication between React, Node.js and MongoDB.

5. Why MongoDB (NoSQL) for ShopEase

Point	Relational (SQL) database	MongoDB (document database)
Data model	Tables with fixed rows and columns	Collections of flexible JSON-like documents
Schema	Fixed; changed through migrations	Flexible; structure defined in the application with Mongoose
Relationships	Foreign keys and JOINs	References between documents, or fields embedded in a document
Fit with ShopEase	Objects must be mapped to tables	Documents match the JSON used by React and Express
Product data	Different attributes need extra columns or tables	Each product is one self-contained document
Scaling	Mainly vertical	Replication and horizontal scaling (sharding)

For ShopEase, MongoDB suits the product catalogue and the orders: an order, together with its items and delivery details, is naturally read and written as one document. A relational database would be the stronger choice where complex multi-table transactions and heavy joins dominate.

6. System Overview and Role of the Database

The application follows a three-layer architecture:

```
React.js Frontend  ->  REST API  ->  Node.js / Express Backend  ->  Mongoose  ->  MongoDB
```

The frontend sends HTTP requests to backend endpoints. Express routes validate and process the requests, Mongoose communicates with MongoDB, and JSON responses are returned to the React application. The frontend never connects to the database directly, so every read and write passes through the API, where authentication and authorization are checked.

7. Data Requirements

Data	What is stored	Used by
Users	Name, email, password, role and wishlist	Registration, login, wishlist, authorization
Products	Name, price, category, image, description and stock	Catalogue, product details, cart, admin management
Orders	User, items, customer details, payment method, total and status	Checkout, order history, admin order management

The database must answer questions such as: does a user with this email exist and does the password match; which products belong to a category and how much stock is left; which orders belong to the logged-in user; and which orders exist in each status and what is the total sales amount.

8. Conceptual Design

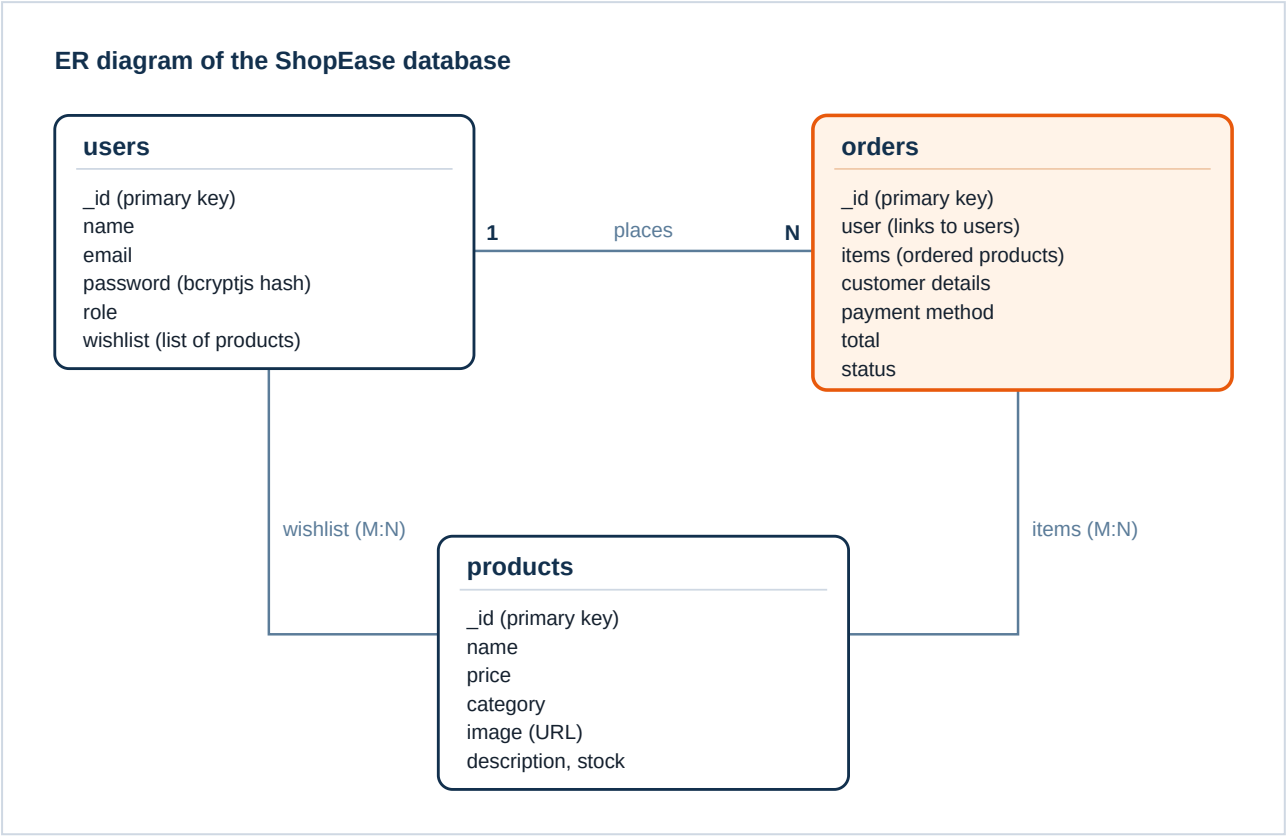
8.1 Entities

The three main entities are **User**, **Product** and **Order**. Each is stored as a collection, and every document in a collection has a unique `_id` generated by MongoDB.

8.2 Relationships

Relationship	Cardinality	Explanation
User places Order	One to many (1:N)	One user can place many orders. Each order carries a user field that links it to its customer.
Order contains Product	Many to many (M:N)	An order holds many products in its items, and a product can appear in many orders.
User saves Product (wishlist)	Many to many (M:N)	A user keeps a wishlist of products, and a product can be on many users' wishlists.

8.3 ER Diagram



One user places many orders (1:N). Users and orders each connect to products through a list: the wishlist in users and the items in orders (M:N).

9. Logical Design: Collections and Fields

The project uses MongoDB, and Mongoose schemas define the fields of each collection.

9.1 users collection

Field	Description
_id	Unique identifier (primary key) generated by MongoDB
name	Full name of the user
email	Email address used to log in
password	Password, stored as a bcryptjs hash and never as plain text
role	Role of the user, used to separate customers from administrators
wishlist	Products saved by the user

9.2 products collection

Field	Description
_id	Unique identifier (primary key)
name	Product name
price	Price of the product
category	Product category (Fashion, Electronics, Shoes, Accessories)
image	Image URL of the product
description	Short description of the product
stock	Available quantity

9.3 orders collection

Field	Description
_id	Unique identifier (primary key)
user	Link to the user who placed the order
items	Products ordered, with their quantities
customer details	Delivery information entered at checkout: full name, email, mobile number, city, pincode and delivery address
payment method	Payment option chosen at checkout (Cash on Delivery is offered in the checkout form)
total	Total amount of the order
status	Current status of the order, updated by the administrator

10. References and Embedded Fields

Data	Approach	Reason
Items and customer details of an order	Stored as fields of the order document	They are always needed together with the order, so one read returns the full order.
Order to user	Link through the user field	A user has many orders, so user data is kept once in users and not copied into every order.
Wishlist	List of products saved in the user	Product data stays in one place, in the products collection.

This combination gives quick reads for the most common screens, such as My Orders, while avoiding duplicate copies of users and products.

11. Normalization and Denormalization

Relational normalization (1NF, 2NF, 3NF) removes repeated and dependent data by splitting it into separate tables. ShopEase applies the same idea in MongoDB:

- **Separate collections** for users, products and orders keep each fact in one place, which is the purpose of 3NF.
- **No repeating column groups:** the items of an order are held as a list inside the order, which is the document-database way to store a one-to-many list.
- **Controlled denormalization:** delivery details and ordered items are kept inside the order, trading a little duplication for one-step reads of an order.

12. Data Integrity and Access Control

MongoDB does not enforce foreign keys, so integrity is maintained by the application layer:

Mechanism	How it protects the data
Mongoose schemas	Define the fields of each collection and keep documents in a consistent structure.
Express route validation	Routes validate and process each request before it reaches the database.
JWT authentication	Only logged-in users can call protected endpoints.
Authorization middleware	Admin-only operations (product changes, all orders, order status) are protected.
bcryptjs hashing	Passwords are stored only as hashes.

13. CRUD Operations Mapped to the Application

Every REST endpoint of ShopEase ends in a database operation:

Application action	Endpoint	CRUD	Database operation
Register user	POST /api/auth/register	Create	Insert a document into users
Login	POST /api/auth/login	Read	Find the user and check the password hash
View all products	GET /api/products	Read	Read documents from products
View one product	GET /api/products/:id	Read	Read one product by its id
Add product (admin)	POST /api/products	Create	Insert a document into products
Update product (admin)	PUT /api/products/:id	Update	Update one product by its id
Delete product (admin)	DELETE /api/products/:id	Delete	Delete one product by its id
Place order	POST /api/orders	Create	Insert a document into orders
My orders	GET /api/orders/my	Read	Read orders of the current user
All orders (admin)	GET /api/orders	Read	Read all documents from orders
Update order status (admin)	PUT /api/orders/:id/status	Update	Update the status field of one order
View wishlist	GET /api/wishlist	Read	Read the user's wishlist products
Toggle wishlist item	POST /api/wishlist	Update	Add or remove a product in the user's wishlist

Sample queries on the ShopEase data model

The queries below use the fields listed in Section 9 and can be run in the MongoDB shell on the ShopEase database.

```
// Read: products of one category, cheapest first
db.products.find({ category: "Electronics" }).sort({ price: 1 })

// Read: products that are in stock
db.products.find({ stock: { $gt: 0 } })

// Read: products priced between 1000 and 5000
db.products.find({ price: { $gte: 1000, $lte: 5000 } })

// Update: change the stock of one product
db.products.updateOne({ name: "Smartphone" }, { $set: { stock: 8 } })

// Delete: remove one product
db.products.deleteOne({ name: "Smartphone" })

// Read: orders of one user (userId = the _id of that user)
db.orders.find({ user: userId })
```

14. Aggregation and Reporting Queries

The aggregation framework passes documents through stages such as `$group`, `$sort` and `$lookup`, and is used for reports. These pipelines work on the fields of ShopEase's collections.

```
// 1. Number of orders in each status
db.orders.aggregate([
  { $group: { _id: "$status", totalOrders: { $sum: 1 } } }
])

// 2. Total sales amount of all orders
db.orders.aggregate([
  { $group: { _id: null, totalSales: { $sum: "$total" } } }
])

// 3. Number of products and average price in each category
db.products.aggregate([
  { $group: { _id: "$category", count: { $sum: 1 }, avgPrice: { $avg: "$price" } }
},
  { $sort: { count: -1 } }
])

// 4. Amount spent and number of orders for each user
db.orders.aggregate([
  { $group: { _id: "$user", spent: { $sum: "$total" }, orders: { $sum: 1 } } },
  { $sort: { spent: -1 } }
])

// 5. Orders together with customer details (a JOIN using $lookup)
db.orders.aggregate([
  { $lookup: { from: "users", localField: "user", foreignField: "_id", as:
"customer" } },
  { $unwind: "$customer" },
  { $project: { total: 1, status: 1, "customer.name": 1, "customer.email": 1 } }
])
```

Query 1 helps the administrator see how many orders wait for action, query 2 gives the sales total, query 3 summarizes the catalogue, query 4 identifies the best customers, and query 5 shows how MongoDB joins two collections when a combined view is needed.

15. Indexing and Query Performance

An index lets MongoDB find documents without scanning the whole collection. MongoDB automatically keeps an index on `_id`. ShopEase runs a few queries again and again, and the following indexes are the recommended design to support them:

Collection	Index	Query it supports
users	email (unique)	Finding a user at login and preventing duplicate accounts
products	category	Category filter on the products page
products	name (text index)	Product search box
orders	user	My Orders for the logged-in user
orders	status	Admin filtering of orders by status

```
db.users.createIndex({ email: 1 }, { unique: true })
db.products.createIndex({ category: 1 })
db.products.createIndex({ name: "text" })
db.orders.createIndex({ user: 1 })
db.orders.createIndex({ status: 1 })

// Check whether a query uses an index
db.orders.find({ user: userId }).explain("executionStats")
```

For a small catalogue the benefit is small, but as products and orders grow, these indexes keep the common screens fast.

16. Transactions and Concurrency

MongoDB guarantees that a write to a single document is atomic. Because ShopEase stores an order, with its items and customer details, as one document, inserting an order either succeeds completely or does not happen at all.

When a change must update several documents together, such as saving an order and also reducing product stock, MongoDB offers multi-document transactions (available on replica sets), which give ACID behaviour across documents. For keeping stock correct when several customers buy at the same time, a conditional atomic update is used so that two customers cannot buy the last item together:

```
// Reduce stock only if enough stock exists (atomic, no race condition)
db.products.updateOne(
  { _id: productId, stock: { $gte: qty } },
  { $inc: { stock: -qty } }
)
```

This approach, together with transactions, is the recommended way to extend ShopEase for order placement under heavy use (see Future Scope).

17. Security of the Data

- Passwords are hashed using bcryptjs before storage.
- JWT tokens are used for authenticated requests.
- Admin-only operations are protected by authorization middleware.
- Environment variables are used for the database and JWT configuration, so secrets are not written in the source code.

- The frontend has no direct access to MongoDB; all access goes through the REST API.

18. Backup, Recovery and Scalability

- **Backup and restore:** `mongodump` and `mongorestore` export and import the contents of the database.
- **Replication:** a replica set keeps copies of the data on several servers for availability.
- **Sharding:** large collections can be split across servers to scale horizontally.

Production deployment and cloud database configuration are outside the academic scope of this project, but the data model does not prevent any of them.

19. Testing of Database Operations

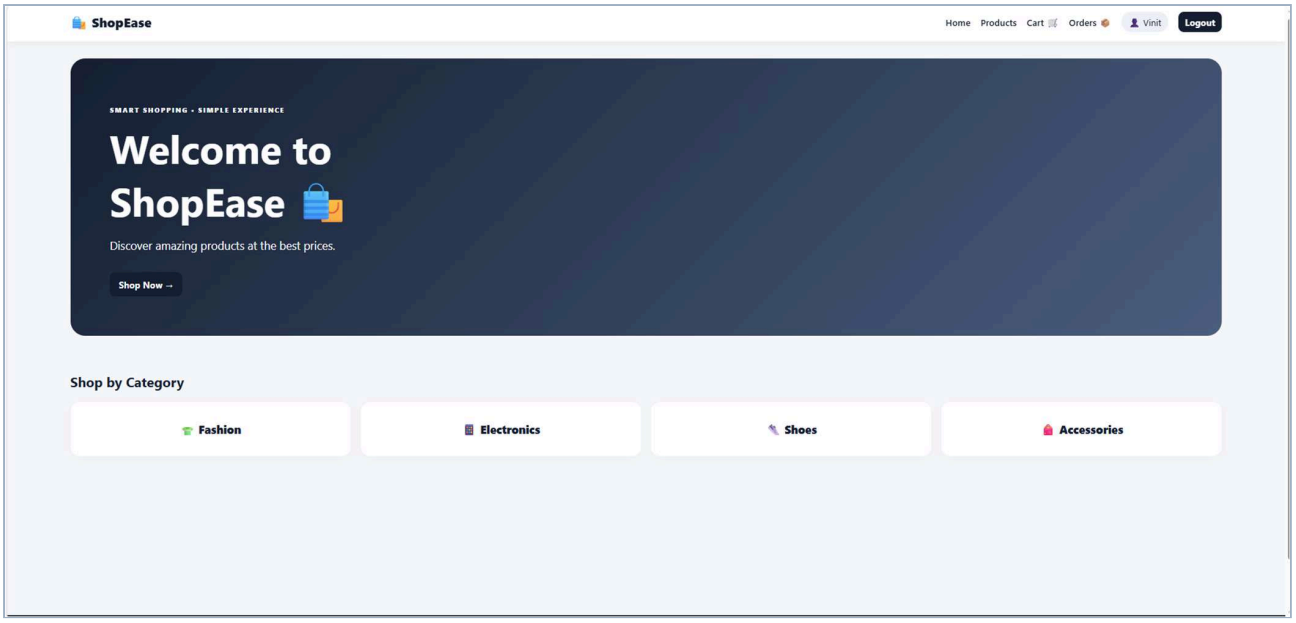
No.	Test case	Database operation tested	Expected result	Status
1	User registration	Insert into users	Account created successfully	Pass
2	User login	Find user and check password	JWT token returned	Pass
3	Product listing	Read from products	Products displayed from MongoDB	Pass
4	Product details	Read one product	Selected product displayed	Pass
5	Cart operations	Quantity and total calculation	Quantity/total updated correctly	Pass
6	Checkout	Insert into orders	Order created in database	Pass
7	Order history	Read orders of the user	User's orders displayed	Pass
8	Admin product operation	Insert, update, delete in products	Authorized admin can manage products	Pass
9	Order status update	Update status in orders	Admin can update order status	Pass

All nine test cases passed.

20. Application Screenshots

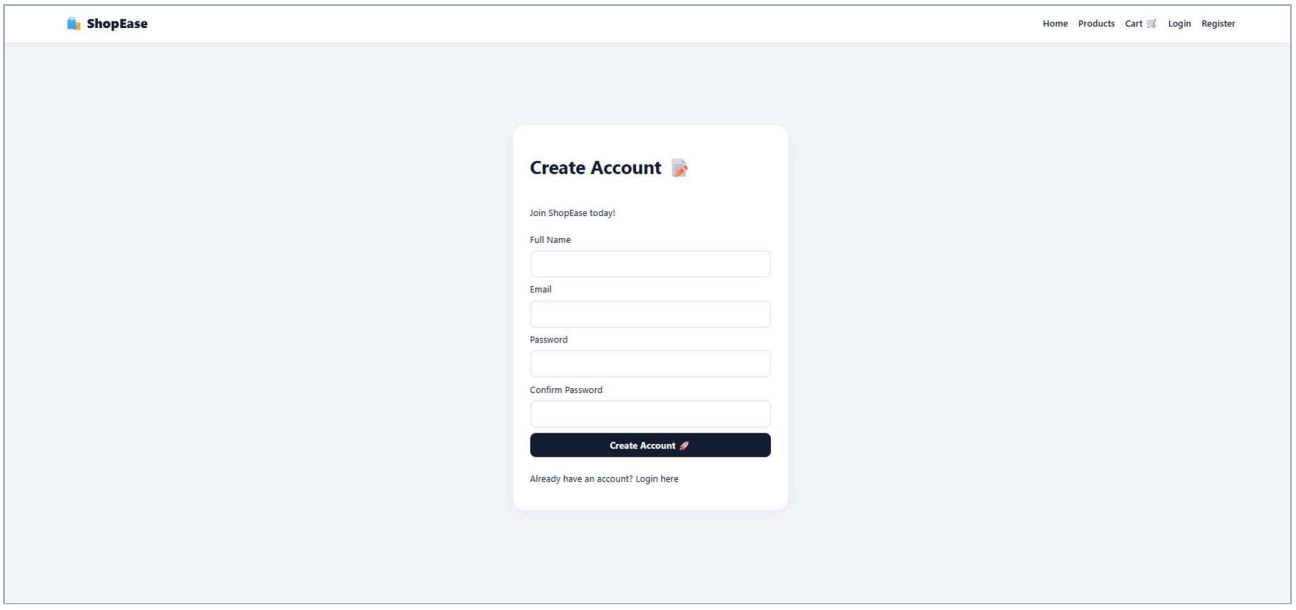
The following screens of the running ShopEase application show the features that read from and write to the database.

Figure 1: Home Page



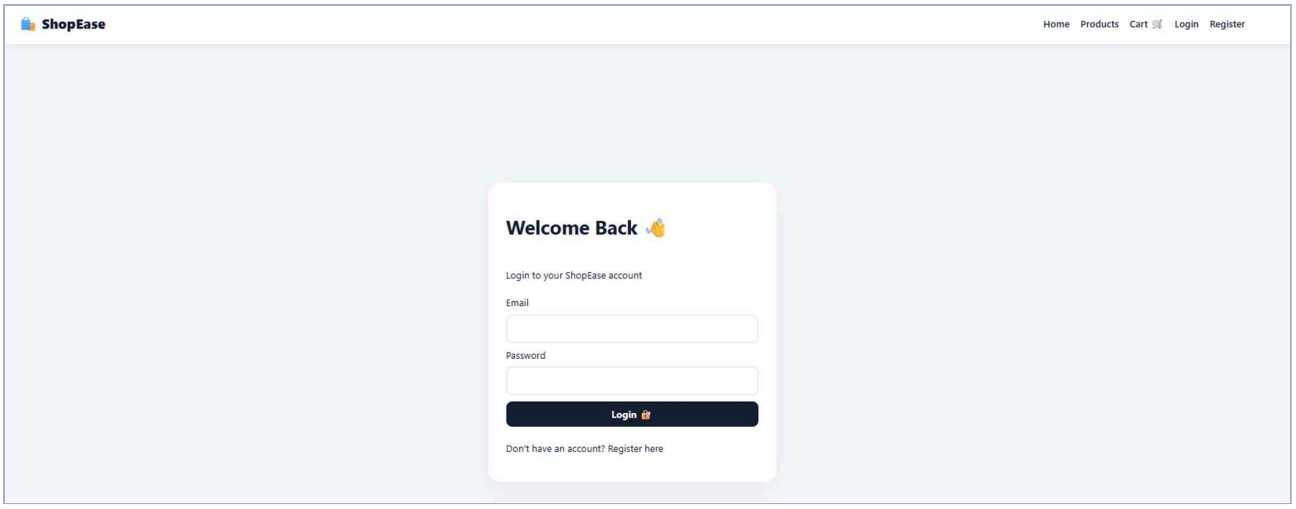
ShopEase home page with the navigation bar and the product categories (Fashion, Electronics, Shoes, Accessories). Categories come from the category field of the products collection.

Figure 2: Register Page



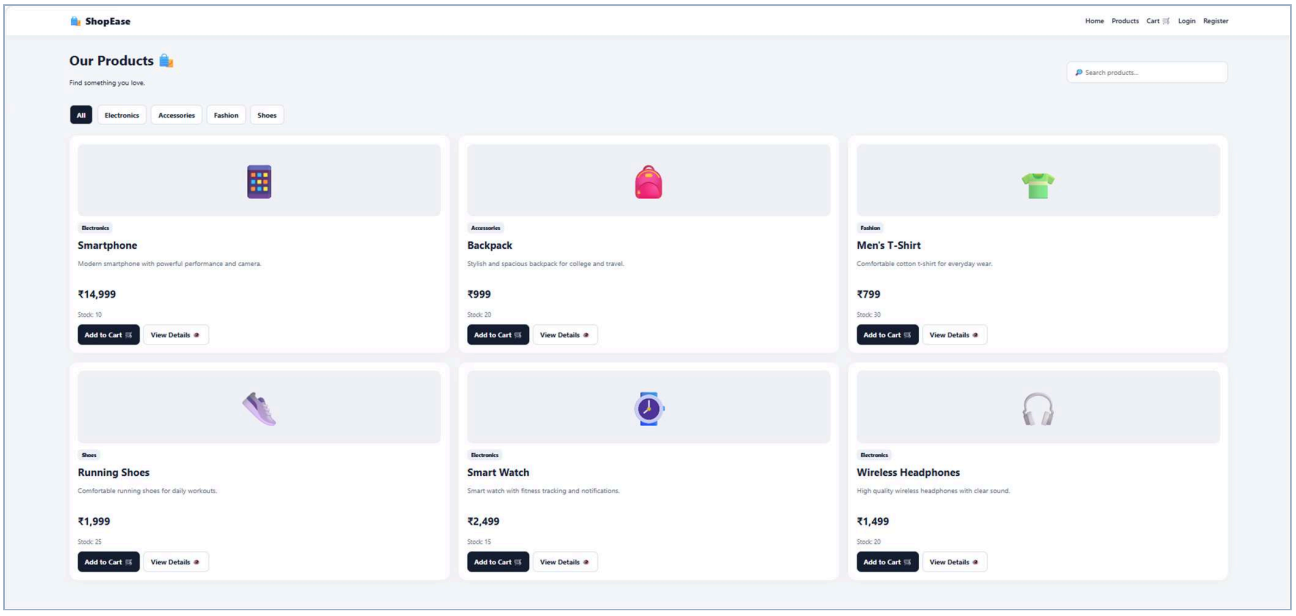
User registration form. Submitting it inserts a new document into the users collection (POST /api/auth/register).

Figure 3: Login Page



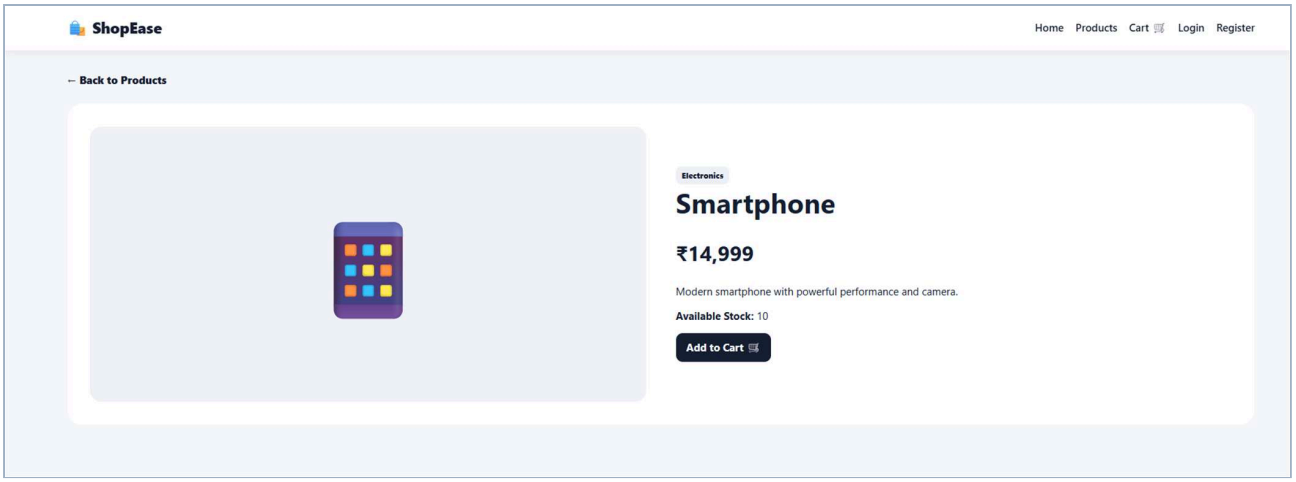
User login form. The email and password are checked against the users collection and a JWT token is returned (POST /api/auth/login).

Figure 4: Products Page



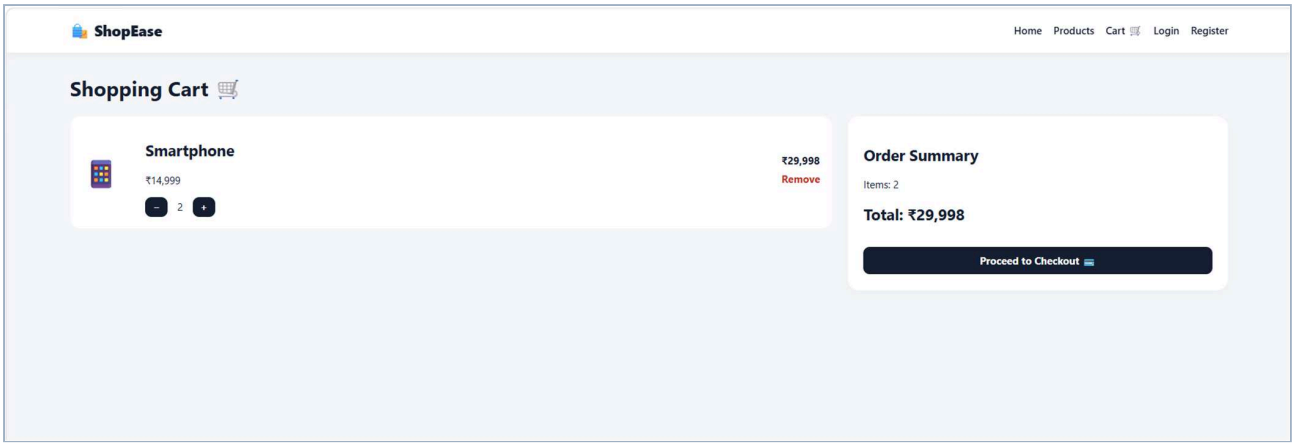
Product catalogue read from the products collection, showing name, category, price and stock (GET /api/products).

Figure 5: Product Details



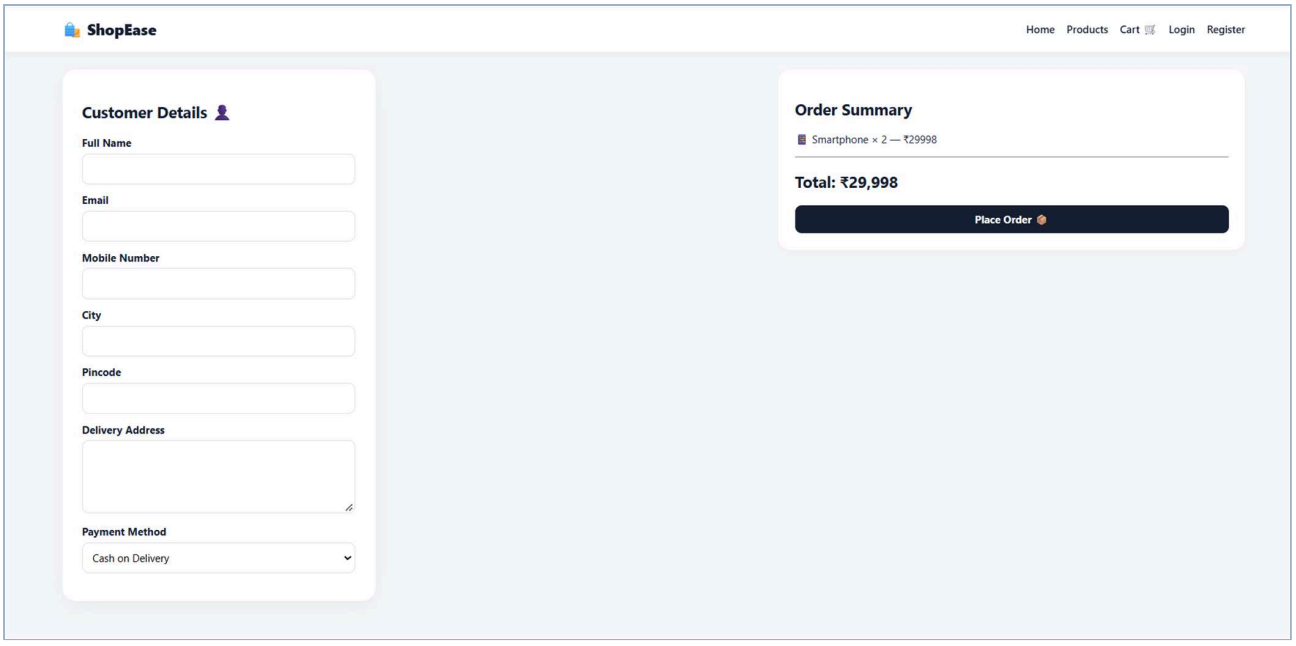
Details of one product, read from the products collection by its id (GET /api/products/:id).

Figure 6: Shopping Cart



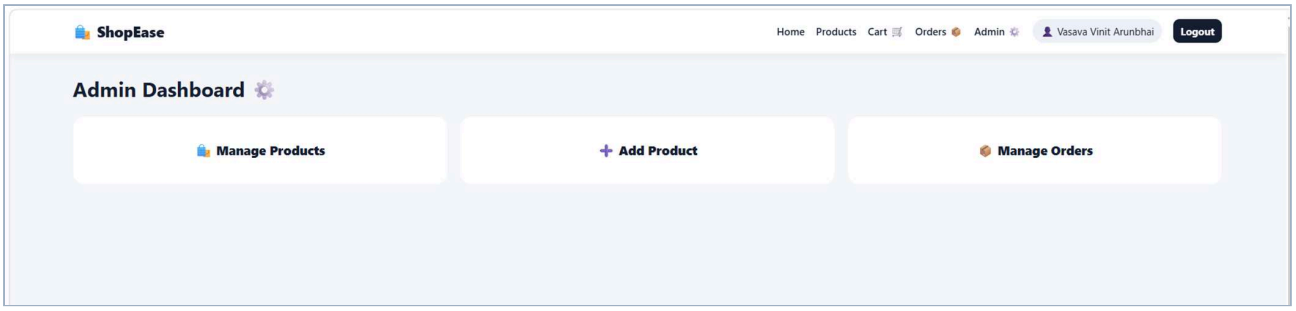
Selected product, quantity and total amount (Rs. 14,999 x 2 = Rs. 29,998) before checkout.

Figure 7: Checkout Page



Customer and delivery details with the order summary. Placing the order inserts a document into the orders collection (POST /api/orders).

Figure 8: Admin Dashboard



Administrator dashboard with Manage Products, Add Product and Manage Orders. These actions change the products and orders collections.

21. Results

The completed application provides an end-to-end e-commerce workflow on top of the database. The React frontend communicates successfully with the Node.js/Express backend, while MongoDB stores the application data. Authentication, product browsing, cart, checkout, orders, wishlist and administrative operations are implemented as integrated modules, and all nine database-related test cases passed.

22. Limitations

- Payment processing is not connected to a real payment gateway.
- Product images are stored as image URLs rather than uploaded files.
- Advanced recommendation and analytics features are not included.
- Production deployment and cloud database configuration are outside the academic scope.

23. Future Scope

- Create and tune indexes for the frequent queries (login, category filter, My Orders) and verify them with explain().
- Use multi-document transactions and conditional atomic updates for order placement and stock update.
- Build an admin sales dashboard using aggregation pipelines, and add downloadable invoices.
- Host the database on a cloud service such as MongoDB Atlas with automatic backups and replication.
- Integrate Razorpay/Stripe or another payment gateway.
- Add product image upload with cloud storage.
- Add product reviews and ratings.
- Add advanced search, filtering and recommendation features.

24. Conclusion

ShopEase shows how a document database can support a complete e-commerce workflow. Users, products and orders are modelled as separate collections, relationships are handled through links and lists, and every application feature maps to a clear database operation. With schema definition, role-based access and password hashing in place, and with indexing, transactions and aggregation as clear extensions, the project demonstrates the main ideas of advanced database management using MongoDB. It provides a practical foundation that can be extended with online payments, analytics, reviews and cloud deployment.

25. References

- MongoDB Manual: https://www.mongodb.com/docs/manual
- MongoDB Aggregation: https://www.mongodb.com/docs/manual/aggregation
- MongoDB Indexes: https://www.mongodb.com/docs/manual/indexes
- MongoDB Transactions: https://www.mongodb.com/docs/manual/core/transactions
- Mongoose documentation: https://mongoosejs.com/docs
- Express.js documentation: https://expressjs.com
- React documentation: https://react.dev
- ShopEase Project Report, Web Application Development (WAD), L.D College of Engineering, 2025-2026